pfi

Python for Fixed Income

COURSE 1 · PYTHON FOUNDATIONS

Introduction to Python, Part 2

WEEK 1, DAY 1, PART 2

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

By the end of this part you can

Make a decision

Premium or discount, investment grade or high yield.

```
if price > 100:
```

Repeat a calculation

Once for every bond, without retyping it.

```
for price in price_list:
```

Build a list in one line

The same loop, written as a list comprehension.

```
[f * c / 100 for f, c in pairs]
```

Write your own function

Define it once. Use it anywhere.

```
def current_yield(coupon, price):
```

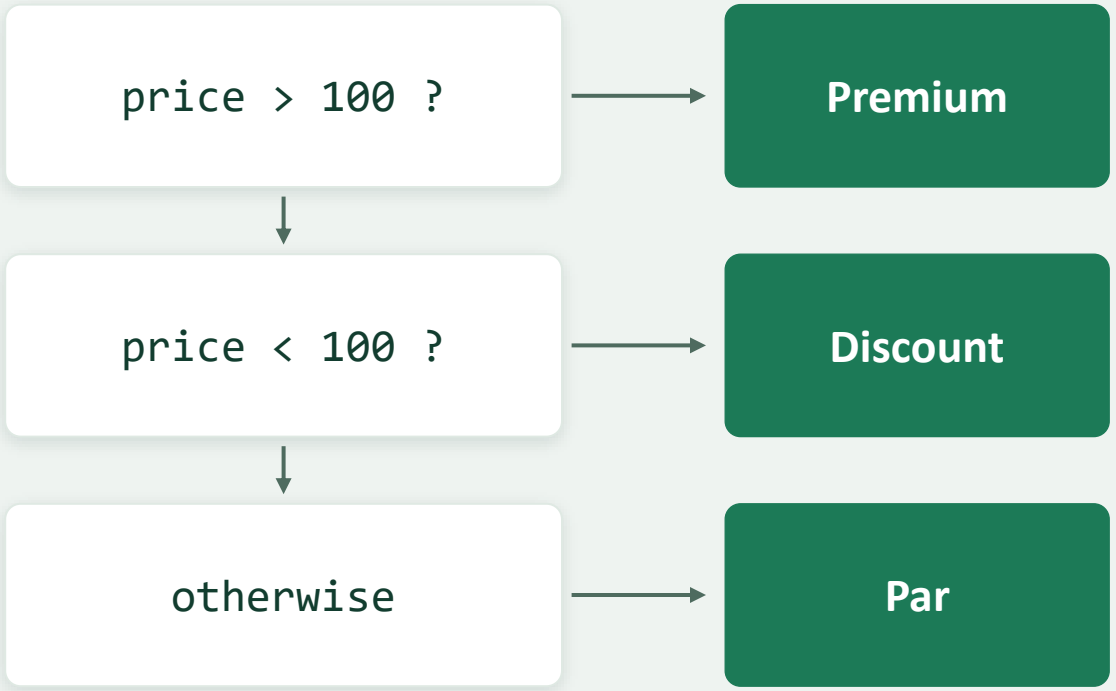
The operators that compare

Each one gives True or False, and each has an Excel form

Python	Meaning	Excel
`==`	equal to	`=`
`!=`	not equal to	`<>`
`<  <=`	less than, or equal to	`<  <=`
`>  >=`	greater than, or equal to	`>  >=`
`in, not in`	one of the items, or not	`COUNTIF (below)`
`and`	both sides are true	`=AND(A1>10,B1>100)`
`or`	at least one side is true	`=OR(A1<=10,B1>100)`
`not`	flips True and False	`=NOT(A1<=10)`

```
'DNMR' in ticker_list  is  =COUNTIF(A1:A4,"DNMR")>0
```

Making a decision




```
if price > 100:
    print('Premium')
elif price < 100:
    print('Discount')
else:
    print('Par')
```

Each test ends with a colon. The indented lines belong to it.

In Excel `=IF(A1>100,"Premium",IF(A1<100,"Discount","Par"))`

A loop is the fill-down

In Excel: one formula, dragged down the column

Face	Price	Cost
1,500,000	98.50	=A2*B2/100
500,000	103.25	=A3*B3/100
750,000	95.00	=A4*B4/100
250,000	101.00	=A5*B5/100

In Python: one formula, applied to every row

```
for ticker, face, price in zip(ticker_list,
                               face_list,
                               price_list):
    cost = face * price / 100
    print(f'{ticker} cost ${cost:,.2f}')
QVNE cost $1,477,500.00
```

The running total

Start at zero, before the loop → Add to it, inside the loop → Read the answer, after the loop

The same loop in one line

As a loop

```
income_list = []
for face, coupon in pairs:
    income = face * coupon / 100
    income_list.append(income)
```

As a list comprehension

```
income_list = [
    face * coupon / 100
    for face, coupon in pairs
]
```

`[ ]`

The brackets say: build a list

`face * coupon / 100`

The calculation for each item

`for face, coupon in pairs`

Where the items come from

Writing your own function

```
def current_yield(coupon, price):
    """
    Take the coupon (in percent) and the price
    and return the current yield in percent.
    """
    return coupon / price * 100
```

def and a name

Starts the definition. The name is how you call it.

Parameters

The names in the brackets that stand for inputs.

Docstring

Says what goes in and comes out. help() shows it.

return

Hands the answer back.

`print`

Shows the value on screen. Then it is gone.

`return`

Hands the value back, so you can use it next.

Defaults, sorting, and three reading notes

A default in the def line, a keyword argument in the call

```
def total_income(incomes, months=12):
    total = sum(incomes)
    return total * months / 12

total_income(income_list)
Income for a full year: $163,750.00
total_income(income_list, months=6)
Income for six months: $81,875.00
```

Sorting by a value you choose

```
sorted(holdings, key=price_of, reverse=True)
DNMR, HLVF, QVNE, PLWK: highest price first
```

Three things to recognize in other people's code

`key=lambda holding: holding['Price']`

lambda: a one-line function with no name. Take a holding, hand back its price.

`def add_face(*positions):`

One star: any number of values, arriving as a tuple. Usually written *args.

`def net_cash(income, **other_cash):`

Two stars: any number of named values, arriving as a dictionary. Usually written **kwargs.

What you can do now

- Label a bond as premium or discount, investment grade or high yield
- Repeat a calculation across every position with a loop
- Keep a running total, and build a new list in one line
- Write a function and use it again and again

Tomorrow Files and Folders: filing, opening, and reading the desk's daily holdings file